



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 05

NOMBRE COMPLETO: José Santiago Sánchez Medina

N° de Cuenta: 319246881

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 04

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 28 de septiembre del 2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

- ***Importar su modelo de coche propio dentro del escenario a una escala adecuada.***

Modificaciones del código.

```
36      Camera camera;  
37      Model coche_M;  
38      Model capo_M;  
39      Model llantasd_M;  
40      Model llantasi_M;  
41      Model Goddard_M;
```

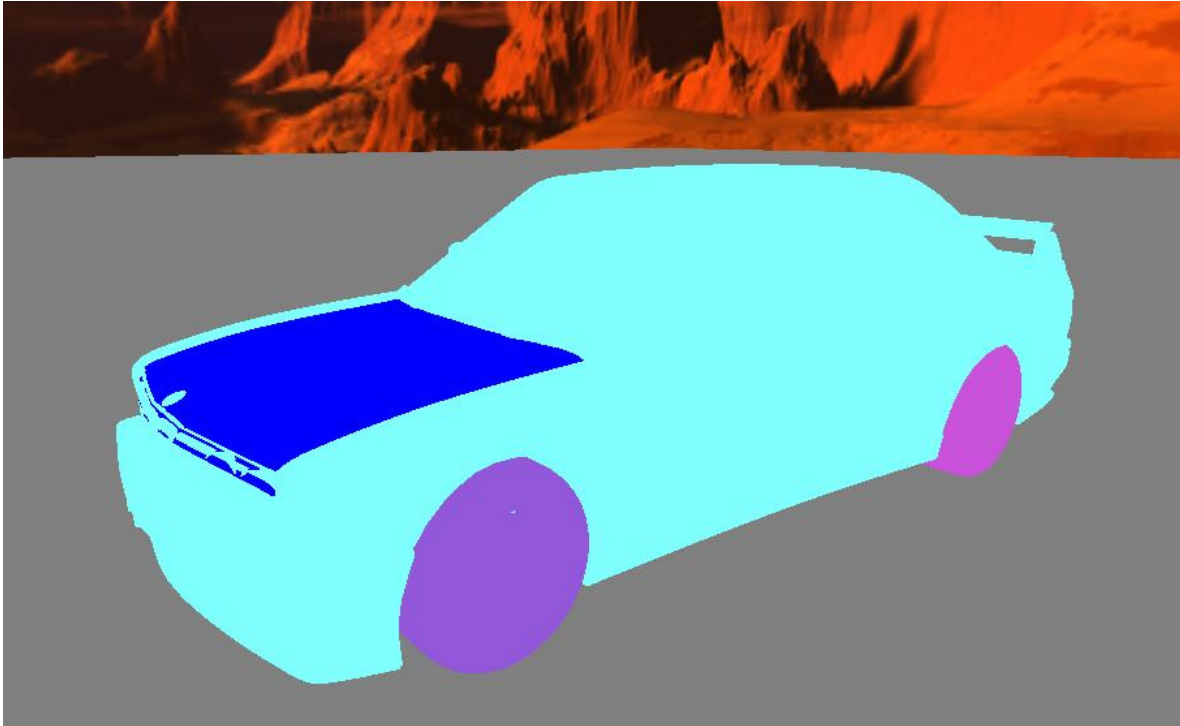
```
coche_M = Model();  
coche_M.LoadModel("Models/coche.fbx");  
capo_M = Model();  
capo_M.LoadModel("Models/capo.fbx");  
llantasd_M = Model();  
llantasd_M.LoadModel("Models/llantasd.fbx");  
llantasi_M = Model();  
llantasi_M.LoadModel("Models/llantasi.fbx");  
Goddard_M = Model();  
Goddard_M.LoadModel("Models/goddarsinmandibula.fbx");
```

```

205 //Coche
206 color = glm::vec3(0.5f, 1.0f, 1.0f);
207 model = glm::mat4(1.0);
208 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
209 model = glm::scale(model, glm::vec3(4.0f));
210 model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
211 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
212 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
213 modelaux = model;
214 coche_M.RenderModel();
215
216 //capo
217 color = glm::vec3(0.0f, 0.0f, 1.0f);
218 model = glm::mat4(1.0);
219 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
220 model = glm::translate(model, glm::vec3(0.0f, 3.5f, 3.7f));
221 model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
222 model = glm::scale(model, glm::vec3(4.0f));
223 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
224 capo_M.RenderModel();
225 modelaux = model;
226
227 //llantas delanteras
228 color = glm::vec3(0.57638230f, 0.3425330f, 0.84923f);
229 model = glm::mat4(1.0);
230 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
231 model = glm::translate(model, glm::vec3(0.0f, 1.2f, 5.3f));
232 model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
233 model = glm::scale(model, glm::vec3(4.0f));
234 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
235 llantasd_M.RenderModel();
236 modelaux = model;
237
238 //llantas traseras
239 color = glm::vec3(0.785883840f, 0.323488420f, 0.849340f);
240 model = glm::mat4(1.0);
241 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
242 model = glm::translate(model, glm::vec3(0.0f, 1.2f, -5.0f));
243 model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
244 model = glm::scale(model, glm::vec3(4.0f));
245 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
246 llantasi_M.RenderModel();
247 modelaux = model;

```

Ejecución



Lo que hice fue hacer unas sencillas modificaciones al código para poder añadir el coche, lo exporté en. fbx tanto el coche sin capo y llantas, como los objetos anteriormente mencionados, ya no tuve problemas en acomodarlo y me resulto sencillo.

- ***Importar sus 4 llantas y acomodarlas jerárquicamente, agregar el mismo valor de rotación a las llantas para que al presionar puedan rotar hacia adelante y hacia atrás.***
- ***Importar el cofre del coche, acomodarlo jerárquicamente y agregar la rotación para poder abrir y cerrar.***
- ***Agregar traslación con teclado para que pueda avanzar y retroceder de forma independiente***

Modificación

```

//coche
color = glm::vec3(0.5f, 1.0f, 1.0f);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, mainWindow.getMovCoche()));
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(4.0f));
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
coche_M.RenderModel();

//Capo
color = glm::vec3(0.0f, 0.0f, 1.0f);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, mainWindow.getMovCoche()));
model = glm::translate(model, glm::vec3(0.0f, 3.5f, 3.7f));
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(4.0f));
model = glm::rotate(model, glm::radians(mainWindow.getRotCofre()), glm::vec3(-1.0f, 0.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
capo_M.RenderModel();

//llantas delanteras
color = glm::vec3(0.57638230f, 0.3425330f, 0.84923f);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, mainWindow.getMovCoche()));
model = glm::translate(model, glm::vec3(0.0f, 1.2f, 5.3f));
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(4.0f));
model = glm::rotate(model, glm::radians(mainWindow.getRotLlantas()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
llantasd_M.RenderModel();

//llantas traseras
color = glm::vec3(0.785883840f, 0.323488420f, 0.849340f);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, mainWindow.getMovCoche()));
model = glm::translate(model, glm::vec3(0.0f, 1.2f, -5.0f));
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(4.0f));
model = glm::rotate(model, glm::radians(mainWindow.getRotLlantas()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
llantas1_M.RenderModel();

```

```
Window.h  Window.cpp  practica5.cpp
Project5  (Ámbito

1  #pragma once
2  #include<stdio.h>
3  #include<glew.h>
4  #include<glfw3.h>
5
6  class Window
7  {
8  public:
9      Window();
10     Window(GLint windowHeight, GLint windowHeight);
11     int Initialise();
12     GLfloat getBufferWidth() { return bufferWidth; }
13     GLfloat getBufferHeight() { return bufferHeight; }
14     bool getShouldClose() {
15         return glfwWindowShouldClose(mainWindow);
16     }
17     bool* getsKeys() { return keys; }
18     GLfloat getXChange();
19     GLfloat getYChange();
20     void swapBuffers() { return glfwSwapBuffers(mainWindow); }
21
22     //Coche
23     GLfloat getMovCoche() { return movCoche; } // Traslación
24     GLfloat getRotLlantas() { return rotLlantas; } // Giro de llantas
25     GLfloat getRotCofre() { return rotCofre; } // Apertura cofre
26
27     ~Window();
28 private:
29     GLFWwindow *mainWindow;
30     GLint width, height;
31
32     //variables coche
33     GLfloat movCoche;
34     GLfloat rotLlantas;
35     GLfloat rotCofre;
36     bool keys[1024];
37     GLint bufferWidth, bufferHeight;
38     GLfloat lastX;
39     GLfloat lastY;
40     GLfloat xChange;
41     GLfloat yChange;
42     bool mouseFirstMoved;
43     void createCallbacks();
44     static void ManejaTeclado(GLFWwindow* window, int key, int code, int action, int mode);
45     static void ManejaMouse(GLFWwindow* window, double xPos, double yPos);
46 }
```

Project5

```
1  #include "Window.h"
2
3  ~ Window::Window()
4  {
5      width = 800;
6      height = 600;
7
8      movCoche = 0.0f;
9      rotLlantas = 0.0f;
10     rotCofre = 0.0f;
11
12     for (size_t i = 0; i < 1024; i++)
13     {
14         keys[i] = 0;
15     }
16 }
17 ~ Window::Window(GLint windowHeight, GLint windowWidth)
18 {
19     width = windowHeight;
20     height = windowWidth;
21
22     movCoche = 0.0f;
23     rotLlantas = 0.0f;
24     rotCofre = 0.0f;
25
26     for (size_t i = 0; i < 1024; i++)
27     {
28         keys[i] = 0;
29     }
30 }
```

```

105
106     void Window::ManejaTeclado(GLFWwindow* window, int key, int code, int action, int mode)
107     {
108         Window* theWindow = static_cast<Window*>(glfwGetWindowUserPointer(window));
109
110         if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS)
111         {
112             glfwSetWindowShouldClose(window, GL_TRUE);
113         }
114
115         if (key >= 0 && key < 1024)
116         {
117             if (action == GLFW_PRESS) theWindow->keys[key] = true;
118             else if (action == GLFW_RELEASE) theWindow->keys[key] = false;
119         }
120
121         if (action == GLFW_PRESS || action == GLFW_REPEAT)
122         {
123             // Movimiento coche
124             if (key == GLFW_KEY_E) theWindow->movCoche += 0.1f;
125             if (key == GLFW_KEY_R) theWindow->movCoche -= 0.1f;
126
127             // Rotación llantas
128             if (key == GLFW_KEY_1) theWindow->rotLlantas += 2.0f;
129             if (key == GLFW_KEY_2) theWindow->rotLlantas -= 2.0f;
130
131             // Cofre con límites
132             if (key == GLFW_KEY_3 && theWindow->rotCofre < 45.0f)
133                 theWindow->rotCofre += 2.0f;
134             if (key == GLFW_KEY_4 && theWindow->rotCofre > 0.0f)
135                 theWindow->rotCofre -= 2.0f;
136         }
137     }
138

```



```

/*
//Codigo para mover unicamente el cascaron de manera independiente, sin llantas ni cofre

color = glm::vec3(0.5f, 1.0f, 1.0f);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(4.0f));
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::translate(model, glm::vec3(mainWindow.getMovCoche(), 0.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
coche_M.RenderModel();

color = glm::vec3(0.0f, 0.0f, 1.0f);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 3.5f, 3.7f));
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(4.0f));
model = glm::rotate(model, glm::radians(mainWindow.getRotCofre()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
capo_M.RenderModel();

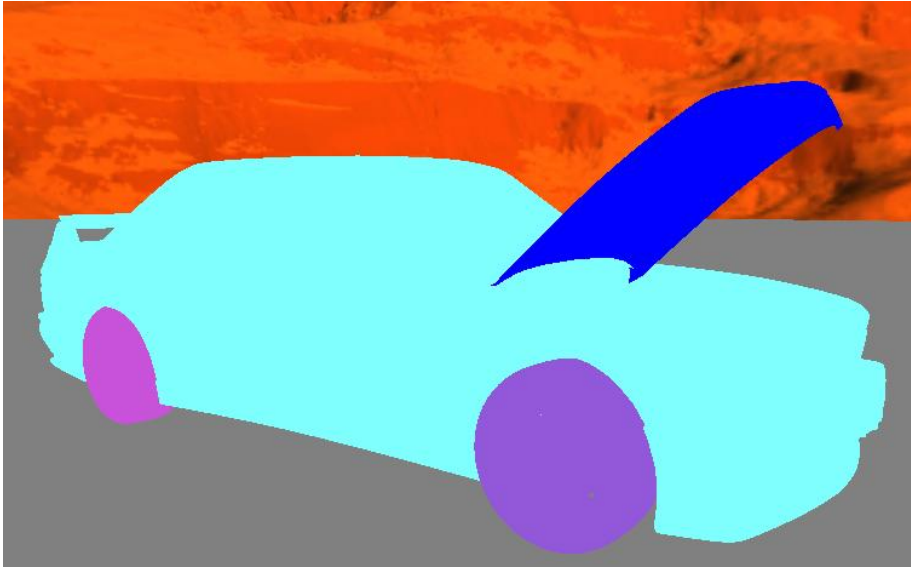
color = glm::vec3(0.57638230f, 0.3425330f, 0.84923f);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 1.2f, 5.3f));
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(4.0f));
model = glm::rotate(model, glm::radians(mainWindow.getRotLlantas()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
llantasd_M.RenderModel();

color = glm::vec3(0.785883840f, 0.323488420f, 0.849340f);
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 1.2f, -5.0f));
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(4.0f));
model = glm::rotate(model, glm::radians(mainWindow.getRotLlantas()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
llantasi_M.RenderModel();
*/

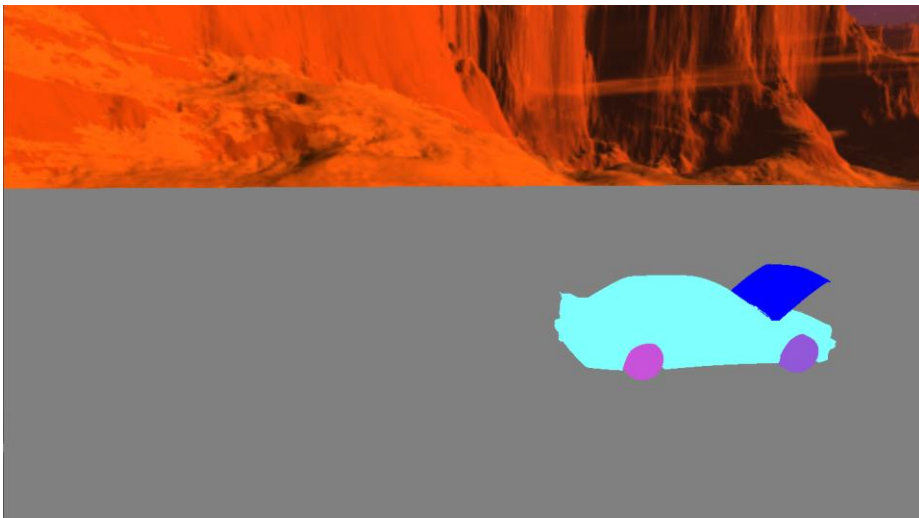
```

Ejecución

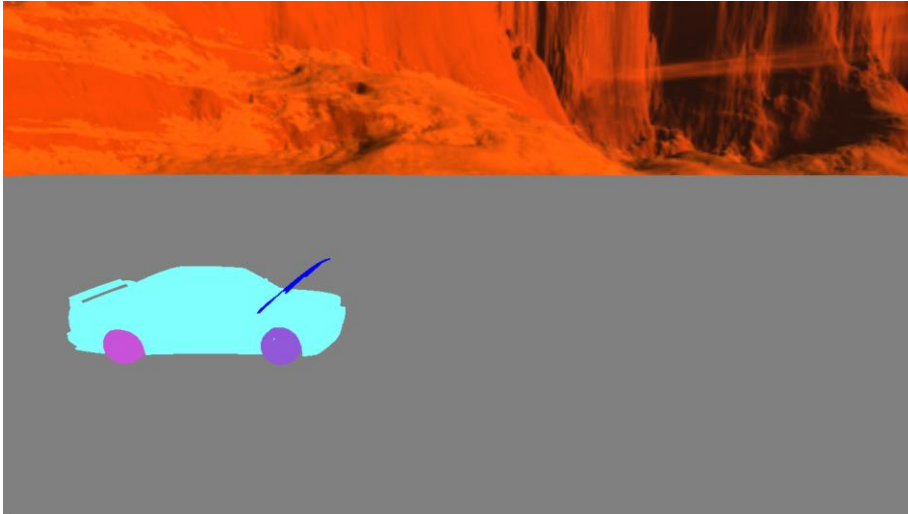
Cofre abierto



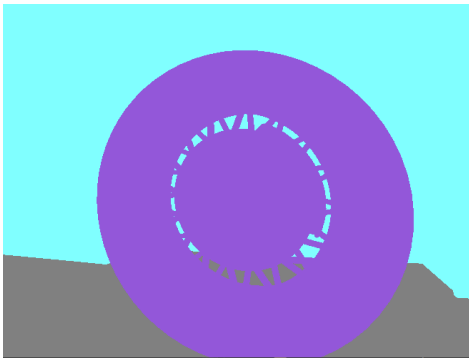
Movimiento delantero



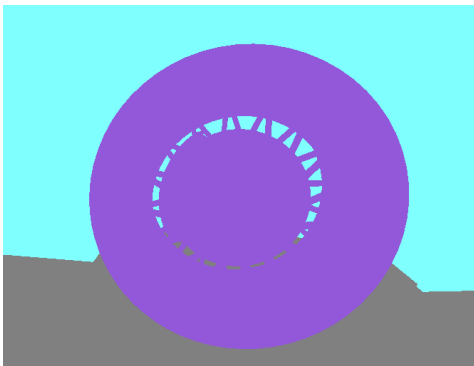
Movimiento trasero



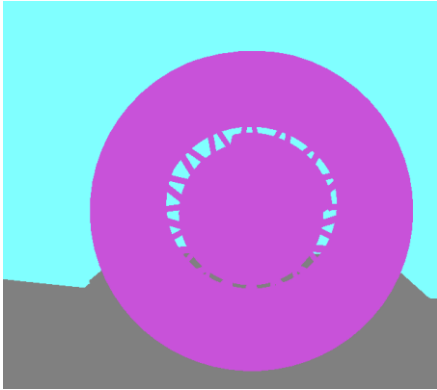
Llantas delanteras hacia adelante



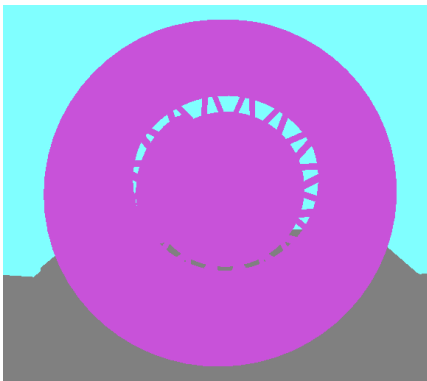
Llantas delanteras hacia atrás



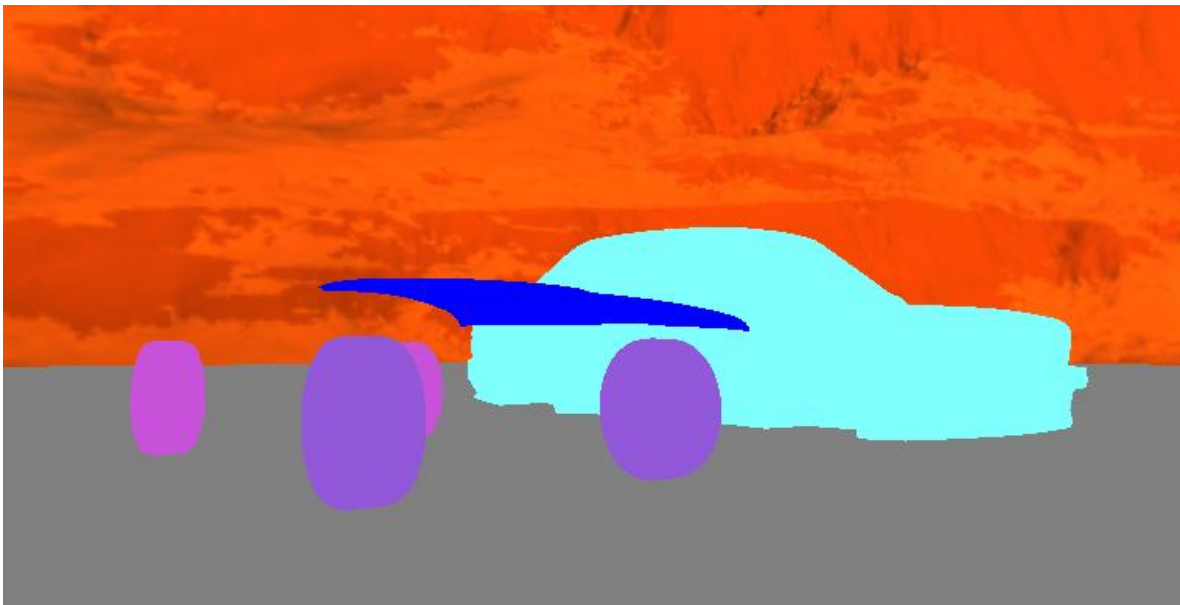
Llantas traseras hacia adelante



Llantas traseras hacia atrás



Movimiento sin llantas ni cofre



Primero tuve el problema de que al exportar mis modelos desde Blender a OpenGL se desacomodaban, porque los ejes no coinciden entre ambos programas. En Blender el eje Z es arriba y en OpenGL normalmente se toma Y como arriba. Al inicio los modelos me aparecían acostados, entonces lo

resolvimos aplicando una rotación en el código para que se vieran bien sin tener que volver a exportar todo desde cero.

Después revisé mi clase Window. Originalmente solo tenía variables de rotación y articulaciones, pero no servía para lo que yo quería: controlar un coche con sus partes. Lo que hice fue modificarla para agregar tres variables nuevas: movCoche para poder trasladar todo el coche, rotLlantas para que las ruedas giren y rotCofre para abrir y cerrar el capó. También adapté el teclado: con E y R avanzo o retrocedo, con 1 y 2 giran las llantas y con 3 y 4 controlo el cofre.

Luego pasé a main.cpp. Yo ya tenía todo bien acomodado manualmente con translate, rotate y scale, y la idea era no perder esos acomodados. Lo que hice fue simplemente sumar las transformaciones dinámicas encima de las fijas: primero la traslación del coche, luego los acomodados que ya tenía, y al final la rotación del cofre o las llantas según cada pieza. De esa manera pude mantener todo en su lugar y al mismo tiempo darle movimiento.

Y agregue un segundo código para mover todo independientemente de las llantas y cofre.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

No presente ningún error, solo me costo acomodar las cosas ya que se importaban viendo para arriba y no de manera horizontal como era necesario.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.

En general, esta práctica me pareció bastante interesante porque pude darle vida a mi modelo del coche, pero también fue más complicada que las anteriores. Lo que más se me complicó al inicio fue entender por qué al exportar desde Blender hacia OpenGL los modelos se desacomodaban, ya que los ejes no coinciden. Resolver eso me tomó tiempo porque tuve que jugar con rotaciones hasta que quedaran igual que en Blender. Después, otro reto fue modificar la clase Window para que no solo manejara entradas básicas, sino que también pudiera controlar las partes del coche como el movimiento completo, las llantas y el cofre.

En el main.cpp lo difícil fue integrar los nuevos movimientos sin perder los acomodados que ya había hecho manualmente, porque si cambiaba el orden de las transformaciones todo se desajustaba. Al final entendí que la clave era mantener mis translate, rotate y scale fijos, y simplemente sumar encima las transformaciones dinámicas que controlo con el teclado. También tuve que corregir el detalle de que el coche se movía de lado en lugar de hacia adelante, lo cual resolví cambiando el eje de traslación.

Creo que fue un buen ejercicio porque me ayudó a entender mejor cómo se construye la jerarquía de un modelo en OpenGL y cómo se pueden combinar transformaciones fijas con dinámicas. Ahora mi coche puede moverse, abrir el cofre y girar las llantas, y lo mejor es que aprendí a controlar cada parte de manera independiente o en conjunto.

b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

Ninguno

c. Conclusión

Me agradó realizar esta práctica porque me permitió aplicar lo aprendido de OpenGL en un modelo más complejo y con varias partes jerárquicas. Al inicio se me dificultó que todo se moviera junto, ya que al trasladar el coche solo se movía el cascarón y las llantas o el cofre quedaban desfasados, pero con pruebas entendí la importancia de aplicar las transformaciones dinámicas en cada pieza además de conservar las transformaciones fijas que ya había definido.

También aprendí a diferenciar entre las posiciones que acomodo manualmente y las que controlo con el teclado, lo cual me dio un panorama más claro de cómo manejar jerarquías en los modelos. Otro detalle que resolví fue el movimiento en el eje equivocado, ya que al inicio el coche se desplazaba lateralmente y logré corregirlo para que avanzara hacia adelante o hacia atrás como un coche real.

Bibliografía en formato APA

Modelo

<https://sketchfab.com/3d-models/free-bmw-m3-e30-ac3c7013434e403e8faff87948caf422>